

01 前缀和与差分

学习目标

- 理解前缀和用于快速求区间和。
- 掌握 `pre[i+1] = pre[i] + a[i]`。
- 掌握区间 `[l, r]` 的和：`pre[r+1] - pre[l]`。
- 初步理解差分：多次区间修改后再还原数组。

核心模型

如果每次都重新累加区间，会重复做很多工作。前缀和先把“从开头到当前位置”的和存起来，再用两次相减得到任意区间和。

```
a = [2, 1, 3, 5, 4]
pre = [0] * (len(a) + 1)
for i in range(len(a)):
    pre[i + 1] = pre[i] + a[i]
```

手算表：

i	a[i]	pre[i]	pre[i+1]
0	2	0	
1	1		
2	3		
3	5		
4	4		

例题1：区间求和

某设备5次采集到的数据为：

```
2 1 3 5 4
```

求第2次到第4次采集数据的总和。若下标从0开始，就是求 `a[1]` 到 `a[3]`。

1. 写出 `pre` 数组：_____。

2. 区间和为 `pre[_____] - pre[_____] = _____`。

参考答案：

1. `pre = [0, 2, 3, 6, 11, 15]`。

2. `pre[4] - pre[1] = 9`。

例题2：代码填空

某温度系统记录了每天是否达标，达标记为1，不达标记为0。现在需要快速查询任意连续几天内达标天数。第 2 天到第 5 天，共有几天达标？

```
ok = [1, 0, 1, 1, 0, 1]
pre = [0] * (len(ok) + 1)
for i in range(len(ok)):
    pre[i + 1] = _____

l = 2
r = 5
ans = _____
print(ans)
```

参考答案：

```
pre[i + 1] = pre[i] + ok[i]
ans = pre[r + 1] - pre[l]
```

输出为 3。

例题3：条件前缀和

统计数组中某区间内偶数个数。

```
a = [3, 6, 2, 5, 8, 7]
pre = [0] * (len(a) + 1)
for i in range(len(a)):
    if a[i] % 2 == 0:
        pre[i + 1] = pre[i] + 1
    else:
        pre[i + 1] = _____

print(pre[5] - pre[1])
```

1. 输出表示哪个区间内的偶数个数? ____。
2. 输出结果: ____。

参考答案:

```
pre[i + 1] = pre[i]
```

`pre[5] - pre[1]` 表示统计 `a[1]` 到 `a[4]` 中的偶数个数, 结果为 3, 对应 6、2、8。

例题4：差分初步

差分可以看成前缀和的逆过程。若 `a` 是真实数组, 差分数组 `diff` 的定义是:

```
diff[0] = a[0]
diff[i] = a[i] - a[i - 1]    # i >= 1
```

因此, 原数组可以由差分数组前缀累加还原:

```
a[i] = diff[0] + diff[1] + ... + diff[i]
```

如果原数组初始全为0, 那么一开始 `diff[i]` 才全部为0。区间修改题常用这种初始状态。

区间 `[l, r]` 整体加 `x` 时, 中间每个位置的值都增加, 但“变化量”只发生在两个边界:

```
diff[l] += x
diff[r + 1] -= x
```

最后从左到右累加 `diff`, 即可还原所有修改后的数组。差分的关键边界是 `r + 1`: 如果 `r` 已经是最后一个下标, 需要保证 `diff` 多开一格, 避免越界。

情景题干:

学校体育馆按0到7共8个时间段统计场地预约数。现在有3个社团提交了预约变更:

```
时间段1到4, 每段增加2个预约
时间段3到7, 每段增加1个预约
时间段0到2, 每段增加3个预约
```

如果逐段修改, 每次都要遍历整个区间。用差分可以只修改区间两端, 最后统一还原每个时间段的预约数。

简化提取后的题干:

长度为8的数组初始全0。执行 $[1,4]+2$ 、 $[3,7]+1$ 、 $[0,2]+3$ 。下标从0开始，求最终数组。

补全代码：

```
n = 8
diff = [0] * (n + 1)
ops = [[1, 4, 2], [3, 7, 1], [0, 2, 3]]
for l, r, x in ops:
    diff[l] += x
    -----

a = [0] * n
cur = 0
for i in range(n):
    cur += -----
    a[i] = cur
```

注意：本题原数组初始全0，所以 `diff` 初始也全0。若原数组一开始不是全0，应先按 `diff[0]=a[0]`、`diff[i]=a[i]-a[i-1]` 构造差分数组。

参考答案：

```
diff[r + 1] -= x
cur += diff[i]
```

最终数组为 $[3, 5, 5, 3, 3, 1, 1, 1]$ 。

当堂限时训练

训练1

使用 `pre[0]=0`、`pre[i+1]=pre[i]+a[i]`。若要枚举所有长度不超过 `k` 的区间 `[l,r]`，下列条件正确的是哪一项？

- A. `r - l <= k`
- B. `r - l + 1 <= k`
- C. `r + l + 1 <= k`
- D. `pre[r] - pre[l] <= k`

参考答案：B。区间长度是 `r-l+1`。

训练2

补全代码，统计 `a[l]` 到 `a[r]` 中大于等于60的个数。

```

a = [58, 72, 61, 49, 80]
pre = [0] * (len(a) + 1)
for i in range(len(a)):
    if a[i] >= 60:
        pre[i + 1] = _____
    else:
        pre[i + 1] = _____

l = 1
r = 4
ans = _____

```

参考答案：

```

pre[i + 1] = pre[i] + 1
pre[i + 1] = pre[i]
ans = pre[r + 1] - pre[l]

```

输出为 3。

训练3

请补全区间和代码。

```

a = [5, 1, 2, 4, 3]
pre = [0] * len(a)
pre[0] = a[0]
for i in range(1, len(a)):
    pre[i] = pre[i - 1] + a[i]

l = 1
r = 3
if l == 0:
    ans = _____
else:
    ans = _____
print(ans)

```

参考答案：

```

if l == 0:
    ans = pre[r]

```

```
else:
    ans = pre[r] - pre[l - 1]
```

这份代码的 `pre` 定义和前面不同，所以区间和不是 `pre[r + 1] - pre[l]`。本题输出为 7。

课后作业

作业1

情景题干：

某运动手环记录了一名学生连续6段训练的体能变化值，正数表示体能状态提升，负数表示疲劳下降：

2, -1, 3, 4, -2, 1

现在要从中选择一段连续训练记录作为“有效训练段”。要求这段训练至少包含2段，最多包含4段，使得这段训练的体能净提升最大。

简化提取后的题干：

给定 `a=[2,-1,3,4,-2,1]`，求长度至少为2、至多为4的连续子数组最大和。

直接双重循环可以枚举 `[l,r]`，但每次重新累加区间会重复计算。先建立前缀和，再用两个端点快速得到区间和。

补全代码：

```
a = [2, -1, 3, 4, -2, 1]
pre = [0] * (len(a) + 1)
for i in range(len(a)):
    pre[i + 1] = _____

best = -10**9
best_l = best_r = -1
for l in range(len(a)):
    for r in range(l, len(a)):
        length = _____
        if _____:
            cur = _____
            if cur > best:
                best = cur
                best_l = l
                best_r = r
```

```
print(best, best_l, best_r)
```

参考答案：

```
pre[i + 1] = pre[i] + a[i]
length = r - l + 1
if 2 <= length <= 4:
    cur = pre[r + 1] - pre[l]
```

本题最大和为 8，对应区间 $[0, 3]$ ，即 $2 + (-1) + 3 + 4$ 。

作业2

情景题干：

学校机房一天分为0到9共10个时段，每个时段最多允许3个班级同时使用。管理员收到若干条预约记录，每条记录为 $[\text{开始时段}, \text{结束时段}]$ ，表示从开始到结束这几个时段都占用1个机房名额。预约记录按进入系统的先后顺序存入队列。现在需要统计每个时段的预约数，并回答某段时间内有多少个时段“超出容量”。

简化提取后的题干：

用队列读出预约区间，用差分统计每个时段预约数，再用条件前缀和统计区间内 $\text{use}[i] > \text{cap}$ 的时段个数。

```
cap = 3
n = 10
q = [[1, 4], [0, 2], [3, 8], [6, 9], [2, 5]]
head = 0
tail = len(q)
diff = [0] * (n + 1)

while head < tail:
    l, r = q[head]
    head += 1
    diff[l] += 1
    diff[r] -= 1
    head += 1

use = [0] * n
cur = 0
for i in range(n):
    cur += diff[i]
    use[i] = cur
```

```

pre = [0] * (n + 1)
for i in range(n):
    pre[i + 1] = pre[i]
    if _____:
        pre[i + 1] += 1

L = 2
R = 7
ans = _____
print(use)
print(ans)

```

填空：

1. 队列非空条件：_____
2. 差分右边界：_____
3. 还原数组：_____
4. 超容量判断：_____
5. 查询 $[L, R]$ 内超容量时段数：_____

参考答案：

```

head < tail
diff[r + 1] -= 1
diff[i]
use[i] > cap
pre[R + 1] - pre[L]

```

$use = [1, 2, 3, 3, 3, 2, 2, 2, 2, 1]$ ，没有超过容量3的时段，所以 $ans = 0$ 。

追问：如果 $cap = 2$ ， $[2, 7]$ 内超容量时段数为多少？

参考答案：3，对应时段 2、3、4。

作业3

某系统记录了每个时间段机房预约数 use ，容量上限为 cap 。现在要判断区间 $[L, R]$ 内是否存在“连续至少2个超容量时段”。下面先构造数组 bad ，其中 $bad[i]=1$ 表示 $use[i] > cap$ ，否则为0。已知：

```

use = [1, 3, 4, 5, 2, 4]
cap = 3
L = 1
R = 5

```


下列程序片段中，能正确完成判断的是哪一项？

A.

```
flag = False
bad = [0] * len(use)
for i in range(len(use)):
    if use[i] >= cap:
        bad[i] = 1
pre = [0] * (len(use) + 1)
for i in range(len(use)):
    pre[i + 1] = pre[i] + bad[i]
for l in range(L, R + 1):
    for r in range(l, R + 1):
        if r - l + 1 >= 2 and pre[r + 1] - pre[l] == r - l + 1:
            flag = True
```

B.

```
flag = False
bad = [0] * len(use)
for i in range(len(use)):
    if use[i] > cap:
        bad[i] = 1
pre = [0] * (len(use) + 1)
for i in range(len(use)):
    pre[i + 1] = pre[i] + bad[i]
for l in range(L, R + 1):
    for r in range(l, R + 1):
        if r - l + 1 >= 2 and pre[r + 1] - pre[l] == r - l + 1:
            flag = True
```

C.

```
flag = False
bad = [0] * len(use)
for i in range(len(use)):
    if use[i] > cap:
        bad[i] = 1
pre = [0] * len(use)
pre[0] = bad[0]
for i in range(1, len(use)):
    pre[i] = pre[i - 1] + bad[i]
for l in range(L, R + 1):
```

```

for r in range(l, R + 1):
    if r - l + 1 >= 2 and pre[r] - pre[l] == r - l + 1:
        flag = True

```

D.

```

flag = False
bad = [0] * len(use)
for i in range(len(use)):
    if use[i] > cap:
        bad[i] = 1
pre = [0] * (len(use) + 1)
for i in range(len(use)):
    pre[i + 1] = pre[i] + bad[i]
for l in range(L, R):
    for r in range(l + 1, R):
        if pre[r + 1] - pre[l] == r - l + 1:
            flag = True

```

参考答案：B。

解析：超容量必须是 `use[i] > cap`，不是 `>=`，所以A错误；B正确，`bad` 表示超容量，若某段 `[l,r]` 的 `bad` 之和等于区间长度，说明整段都超容量；C 使用不多开一格的前缀和，却写成 `pre[r]-pre[l]`，会漏掉左端点；D 的循环 `range(l+1, R)` 漏掉了 `r=R`，不能完全覆盖查询区间。

错题复盘栏

错题	错因	正确方法

教师观察记录

观察点	表现
是否能写出 <code>pre[i+1]</code>	
是否混淆 <code>r</code> 和 <code>r+1</code>	
是否理解条件前缀和	